Ders 6 Genişletilmiş Notu: Faks Sıkıştırma (Grup 3 ve Grup 4, MH ve MR Kodlama), Aritmetik Kodlama ve PPM'e Giriş

Ders 6 Genişletilmiş Notu: Faks Sıkıştırma (Grup 3 ve Grup 4, MH ve MR Kodlama), Aritmetik Kodlama ve PPM’e Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Faks Sıkıştırmaya Giriş ve ITU-T Standartları
- 2. Pels, Scanline ve Çözünürlük
- 3. Sıkıştırmasız Gönderim Süresi Örneği
- 4. Grup 3: Modified Huffman (Tek Boyutlu Kodlama)
- 5. Satır Sonu, Hata Yakalama ve Sayfa Sonu
- 6. İki Boyutlu Kodlama: Modified READ
- 7. İki Boyutlu Kodlamanın Üç Modu
- 8. İki Boyutlu Kodlama Örneği
- 9. Aritmetik Kodlamaya Giriş
- 10. Aralık Daraltma ve İlk Örnek
- 11. SWISS MISS Örneği: Kodlama ve Kod Çözme
- 12. Son Sembol Problemi ve EOF
- 13. Tamsayılarla Aritmetik Kodlama
- 14. PPM: İçerik Tabanlı Sıkıştırmaya Giriş
- 15. Lipogram ve Doğal Dil Dışı Metinler
- 16. PPM’de Order Sayısı ve Context Tablosunun Kurulması
- Örnek Sorular

1. Faks Sıkıştırmaya Giriş ve ITU-T Standartları

Hoca, geçen hafta modemlerde kullanılan sıkıştırma algoritmalarını (MNP5, MNP7) gördüklerini hatırlattı ve bu derste fakslarda kullanılan sıkıştırma algoritmalarını anlatacağını söyledi. Amacı, veri transferi yapan her cihazda, uygulamanın içeriğine ve türüne bağlı olarak farklı algoritmaların geliştirildiğini göstermektir.

> **Hocanın vurgusu:** Hoca, faks ve modem gibi konulardaki ayrıntıları vizede “şunu tasarla, bunun bilmem nesini yap” diye sormayacağını; bu konulara bakış açısını genişletmek için yer verdiğini ve konunun kitapta da bulunduğunu söyledi.

Faksta bir metin de gönderilse bir resim de gönderilse karşı taraf hepsini resim olarak alır; yani metin de aslında görüntü olarak gider. Faks makinelerinde sıkıştırma kullanılmasaydı gönderim süreleri çok uzun olurdu (bu, 3. bölümdeki örnekte görülür).

Slayttaki bilgiler şöyledir (ekrandan, 00:10:00):

- İletişim hattı üzerinden görüntü verisi gönderileceği zaman sıkıştırma önemli hale gelir. Faks makineleriyle görüntünün cihazlar arasında gönderilmesi gündeme gelmiştir.
- ITU-T (International Telecommunication Union) tarafından bir veri sıkıştırma standardı geliştirildi: T2 (Grup 1), T3 (Grup 2).

- Daha sonra bu standartların yerini T4 (Grup 3) ve T6 (Grup 4) aldı.
  - Grup 3 (T4): PSTN (Public Switched Telephone Network, 9600 baud).
  - Grup 4 (T6): ISDN (Integrated Services Digital Network, 64K baud).
- Her iki yöntemde de sıkıştırma oranı 10:1 ya da daha yüksektir.
- Önceden bir sayfanın iletimi yaklaşık 1 dakika sürerken bu standartlarla birkaç saniyeye düşmüştür. Hoca, içerik çok karışıksa sürenin daha da uzadığını ekledi.

Hoca 10:1 oranını "10 pikselin bir pikselle ifade edilmesi" diye düşünebileceğimizi söyledi.

2. Pels, Scanline ve Çözünürlük

Faksta yeni bir kavram karşımıza çıkar: **pels** (picture elements). Bir resimde her nokta bir renge karşılık gelir ve buna piksel denir. Faks makinesinde ise bu noktalara pels denir ve bunlar siyah-beyaz noktalardır.

Hoca "pikselden farkı nedir?" diye sordu ve şöyle açıkladı: Faksta doküman gönderilirken satır satır taranarak gönderilir. Her bir satıra **scanline** denir. Yatay çözünürlük ile dikey çözünürlük birbirinden farklıdır ve dikey çözünürlük seçilen moda göre değişir.

Slayt (ekrandan, 00:14:00):

- Faks makineleri dokümanı satır satır tarar ve satırları "pels" (picture elements) adı verilen küçük siyah-beyaz noktalara çevirir.
- **Yatay çözünürlük**
  - Her zaman 8,05 pels/mm (205 pels/inç) ($1'' = 25{,}4$ mm).
  - 8,5 inç genişliğinde taranan her satır 1738 pels'e çevrilir.
  - T4 standardında tarama için önerilen satır genişliği 8,2 inçtir.
- **Dikey çözünürlük**
  - Standard mode: 3,85 scanline/mm
  - Fine mode: 7,7 scanline/mm
  - Very fine mode: 15,4 scanline/mm

Hoca hesabı şöyle kurdu: 1 inç 25,4 mm'dir; 1 mm'ye 8,05 pels düşüyorsa bunu 25,4 ile çarparak bir inçteki pels sayısı bulunur: $8{,}05 \times 25{,}4 \approx 204{,}5$, yani yaklaşık 205 pels/inç. 8,5 inç genişlik için $8{,}05 \times 8{,}5 \times 25{,}4 \approx 1738$ pels çıkar (python ile doğrulandı).

Dikey çözünürlük kaliteyi belirler. Standart modda kalite biraz düşüktür, bir üstü fine mode, en kalitelisi very fine mode'dur. Kalitenin artması, milimetre başına düşen scanline sayısının artması demektir: very fine modda 1 mm'ye 15,4 scanline sığdırılır, standart modda 3,85. Bu modlar faks makinesinin üzerinde seçimliktir; varsayılan standart moddur.

3. Sıkıştırmasız Gönderim Süresi Örneği

Hoca küçük bir örnek yaptı (ekrandan, 00:17:20): 10 inç yüksekliğindeki bir sayfa sıkıştırma yapılmadan üç farklı modda gönderilirse ne kadar sürer?

10 inç = 254 mm'dir. Her moddaki scanline sayısı yüksekliğin mm başına scanline ile çarpılmasıyla bulunur:

```
254 mm * 3,85  -> standart mode  ->  978 scanline
254 mm * 7,7   -> fine mode      -> 1956 scanline
254 mm * 15,4  -> very fine mode -> 3912 scanline
```

Slayttaki tablo:

```
Scan lines  Pels per line  Pels per page  Time (sec.)  Time (min.)
   978          1664          1.670M          170          2.82
  1956          1664          3.255M          339          5.65
  3912          1664          6.510M          678         11.3
```

Genişlik aynı olduğu için satır başına pels sayısı her modda aynıdır. Kalite arttıkça scanline sayısı ve süre artar. Very fine modda sıkıştırmasız bir sayfa için 11,3 dakika beklemek gerekir. Slaytta "Sürelerdeki uzunluk, faks iletişimde veri sıkıştırmanın ne kadar önemli olduğunu göstermektedir" yazar. Bu süreleri saniyelere indirmek için sıkıştırma algoritmaları kullanılır.

Ders sonrası not: Hoca bir yerde "1 inç 2,54 mm" dedi; doğrusu 25,4 mm'dir ve hesap da buna göre (10 inç = 254 mm) yapıldı. Hoca standart mod için 977 scanline dedi; slaytta 978 yazar ($254 \times 3{,}85 = 977{,}9$).

Ders sonrası not: Slayttaki standart mod pels/sayfa değeri (1.670M) diğer satırlarla tutarsızdır: $978 \times 1664 = 1.627.392$, yani büyük olasılıkla 1.627M olmalıdır. Fine ve very fine satırları tutarlıdır ($1956 \times 1664 \approx 3{,}255$M, $3912 \times 1664 \approx 6{,}510$M). Süreler büyük olasılıkla pels sayısının 9600 bit/sn'ye bölünmesiyle bulunmuştur: $1.627.392/9600 \approx 170$ sn, $6.509.568/9600 \approx 678$ sn (python ile doğrulandı).

4. Grup 3: Modified Huffman (Tek Boyutlu Kodlama)

4.1 Run uzunluklarının analizi

Elimizde bir doküman var ve üzerinde siyah-beyaz pels'ler bulunuyor. Grup 3 algoritması bu siyah ve beyaz değerlerin run değerlerini (arka arkaya kaç tane geldiklerini) analiz eder. Run değerlerine göre önceden hazırlanmış kodlar kullanılır.

Slayt (ekrandan, 00:20:00):

- Grup 3 kodu, birçok doküman üzerinde siyah ve beyaz pels'lerin run-length'lerinin analiz edilmesiyle elde edilir.
- Her bir run uzunluğu için Huffman algoritmasıyla değişken uzunlukta kodlar atanır.
- En fazla kullanılan run uzunluklarının 2, 3 ve 4 siyah pels değerleri olduğu tespit edilmiş ve bu pels değerleri için kısa kodlar atanmıştır.
- 2 ve 7 arasındaki run değerlerine sahip beyaz pels'lere de daha uzun kodlar atanmıştır.
- Grup 3, RLE ve Huffman kodlamanın bir bileşimidir.
- Run değerlerinin uzun olabileceği düşünüldüğünden Huffman algoritması üzerinde değişiklikler yapılmıştır. Bu sebeple Grup 3 standardı **MH (Modified Huffman)** olarak adlandırılır.
- 1 ile 63 run uzunluğundaki pels değerleri için **Termination kodlar**, bu pels değerlerinin 64 katı değerleri için de **Make-up kodlar** kullanılır.

Hoca, bir metin dokümanında siyah parçaların en çok arka arkaya 2, 3 ve 4 kez geldiğinin bulunduğunu ve bunlara daha kısa Huffman kodları atandığını anlattı.

Ders sonrası not: Slayt ve hoca beyaz 2-7 run'lar için "daha uzun kodlar" dedi. Tabloya göre (00:22:30) beyaz 2-7 run'ların kodları 4 bittir (`0111`, `1000`, `1011`, `1100`, `1110`, `1111`) ve beyaz kodların en kısalarıdır. Cümle büyük olasılıkla "bunlara da kısa kodlar atanmıştır" anlamındadır; beyaz kısa kodlar, siyahın en kısa kodlarından (2-3 bit) biraz uzundur.

4.2 Termination ve Make-up tabloları

Tablo şöyle hazırlanmıştır: Yüzlerce farklı içerikli doküman alınır, üzerindeki siyah ve beyaz pels'lerin run'ları sayılır. 0'dan 63'e kadar 64 run değeri için Termination tablosu çıkarılır (ekrandan, 00:22:30). Örneğin:

Run	Beyaz	Siyah
0	00110101	0000110111
1	000111	010
2	0111	11
3	1000	10
4	1011	011
5	1100	0011
6	1110	0010
7	1111	00011
12	001000	0000111

Run değerinin 0 olması nasıl mümkündür? Hoca bunu RLE'deki bir örnekle hatırlattı: Üç tekrardan sonra run değeri yazılan bir yöntemde arka arkaya üç tane 3 geldiğinde run değerine 0 yazılabiliyordu, dört tane 3 geldiğinde 1. Burada da 64'ün tam katı olan bir run için Termination tarafında 0 kodlanır.

Beyazlarda en çok kullanılan 2, 3, 4 run'larına atanan kodlar daha küçüktür ve değerler büyüdükçe kodlar uzar. Siyahlarda ise 2 ve 3 run değerleri dokümanlarda daha çok geçtiği için bunlara daha kısa kodlar atanmıştır.

Make-up tablosu 64'ün katlarından oluşur: 64, 128, 192, … 2560'a kadar. Bu kodlar daha uzundur. Örnekler: 64 beyaz `11011`, 128 beyaz `10010`, 64 siyah `0000001111`, 2560 `000000011111`.

1792'den sonra siyah ve beyaz için aynı kod verilmiştir. Hoca bunu şöyle açıkladı: Koca bir A4 sayfanın ortasında iki satır yazı varsa geri kalanı beyazdır ve arka arkaya gelen run'lar çok uzun olur. Ya da fotokopiden alınmış bir kağıdın kenarları siyah olabilir. Arka arkaya 1792 beyaz ya da 1792 siyah gelmişse bu ya arka plandır ya da bilgi taşımayan bir alandır; bunların ne olarak kodlandığının önemi yoktur.

Tablolar önceden hazırlanır. Faksa yeni bir kağıt konduğunda ilk scanline okunur, siyah ve beyazların run değerleri çıkarılır ve tablodaki kodlar sırayla yazılır.

4.3 Bir run'ın kodlanması

Slayt (ekrandan, 00:26:20): Bir "run" kodu ya tek bir Termination kodundan ya da bir veya daha fazla Make-up kodu ve onu izleyen bir Termination kodundan oluşur.

Termination kodu mutlaka bir tane olmalıdır. Run değerinin 64'e göre durumuna bakılır: 64'ün katı olan kısım Make-up tablosundan, kalan Termination tablosundan alınır. Örneğin 64 beyaz varsa Make-up'tan 64'ün kodu, Termination'dan 0'ın kodu alınır.

Slayttaki örnekler:

```
1. 12 beyaz pels              -> 001000
2. 76 beyaz pels  (64 + 12)   -> 11011 | 001000
3. 140 beyaz pels (128 + 12) -> 10010 | 001000
4. 64 siyah pels  (64 + 0)    -> 0000001111 | 0000110111
5. 2561 siyah pels (2560 + 1)-> 000000011111 | 010
```

- 12 beyaz 64'ün katı değildir; doğrudan Termination tablosundan 12'nin beyaz kodu alınır.
- 76 beyaz: $76 = 64 + 12$. Make-up'tan 64'ün beyaz kodu, Termination'dan 12'nin beyaz kodu.
- 140 beyaz: $140 = 2 \cdot 64 + 12 = 128 + 12$. Make-up'tan 128, Termination'dan 12.
- 64 siyah: Make-up'tan 64, Termination'dan 0.
- 2561 siyah: Make-up'tan 2560, Termination'dan 1.

Bir satırda örneğin 12 beyaz, 3 siyah, 5 beyaz, 2 siyah, 10 beyaz gibi bir dizi gelir; doküman scanline scanline bu şekilde kodlanır.

5. Satır Sonu, Hata Yakalama ve Sayfa Sonu

5.1 EOL ve satır başındaki beyaz pel

Satır satır tarandığına göre satırın bittiği nasıl anlaşılır? Her satırın sonuna bir **EOL (end of line)** kodu konur. Kod çözücü EOL'ü görünce satırın bittiğini anlar ve yeni scanline'ı çözmeye başlar.

Slayt (ekrandan, 00:28:30):

- Her bir tarama satırı ayrı ayrı kodlanır ve kodlar özel EOL koduyla sonlandırılır.
- Bir satır scan edildiğinde her satırın soluna bir tane beyaz pel eklenir. Bu, alıcı tarafından kod çözüldüğünde ortadan belirsizliği kaldırmak içindir.

Örnek: 14 pels'ten oluşan bir satır 3 siyah, 2 beyaz, 2 siyah ve 7 beyazdan oluşuyor. Başına bir beyaz pel eklenir:

```
1w 3b 2w 2b 7w EOL
000111 | 10 | 0111 | 11 | 1111 | EOL
```

Kodlar sırasıyla 1 beyaz (`000111`), 3 siyah (`10`), 2 beyaz (`0111`), 2 siyah (`11`), 7 beyazın (`1111`) Termination kodlarıdır. Satırda 64 ya da daha büyük bir run olmadığı için Make-up kodu kulla-

nılmaz. Hoca, şekilde ilk beyazların görünmediğini, orada aslında iki orijinal beyaz olduğunu ve önlerine bir beyaz daha eklendiğini de gösterdi.

> **Ders sonrası not:** Slaytta EOL kodu `0000000001` olarak okunuyor. ITU-T T.4'te EOL büyük olasılıkla `000000000001` (11 sıfır ve bir 1, toplam 12 bit) olarak tanımlıdır.
>
> **Ek bilgi (derste anlatılmadı):** Satır başına eklenen beyaz pel sayesinde her satır kodu bir beyaz run ile başlar. Satır gerçekte siyahla başlasa bile alıcı ilk kodu her zaman beyaz tablosuna göre çözer; böylece renk sırası (beyaz, siyah, beyaz, …) hiç karışmaz.

5.2 Hata yakalama

Slayt (ekrandan, 00:30:00):

- Grup 3, kodda hata düzeltmeye sahip değildir fakat hataların çoğunu bulabilir.
- Huffman kodun doğasından dolayı iletim anında tek bir hatalı bit gönderilirse bile alıcı senkronizasyonun dışına çıkar ve hatalı bit stringi oluşur.
- Bu sebepten dolayı her satır ayrı ayrı kodlanır. Alıcı hatayı bulmuşsa EOL koduna kadar olan bitleri atlar; böylece bir hata oluştuğunda alıcı tarafında en fazla bir satır düzeltilmeden diğer bilgiler alınır.
- Eğer alıcı satır sayısı kadar EOL kodunu göremezse yüksek bir hata oranı olduğunu kabul eder ve işlemi iptal ederek durumu göndericiye bildirir.
- Kodlar 2 ile 12 bit arasında olduğundan, alıcı 12 biti okuduktan sonra hâlâ geçerli bir kod bulamamışsa hatayı bulur.
- Her sayfanın sonunda 6 tane EOL kodu vardır.
- Bu yöntem **One-Dimensional Coding** olarak adlandırılır.
- Sıkıştırma oranı görüntüye bağlıdır. Text ve B&W görüntülerde sıkıştırma oranı yüksektir.
- Kısa "run"lardan oluşan görüntülerde, özellikle gray scale görüntülerde başarı düşüktür.

Hoca hatanın nasıl bulunduğunu şöyle anlattı: Okuyup okuyup bir türlü EOL'e gelinemiyorsa bir hata vardır. İlk haftalarda anlatıldığı gibi bazen bir bitlik değişim başka bir sembolün koduna denk gelir; o zaman o sembol olarak açılır ve hata fark edilmez. Ama bozulan bitin karşılığına gelen bir kod yoksa hata anlaşılır.

Hatayı anlayan alıcı karşı taraftan yeniden göndermesini istemez; EOL'e kadar gider ve alt satırdan düzgün okumaya devam eder. Hoca bunun büyük bir kayıp olmadığını şöyle açıkladı: Bir karakterin üzerinden belki 30 scanline geçer. Bunlardan biri hatalıysa küçük bir nokta kaybolur ve göz onu zaten tamamlar. Ama arka arkaya 10-15 satır okunamamışsa önemli bilgi kaybolmuş demektir. Alıcı bir satırın genişliğini ve kaç pels içerdiğini bildiği için EOL'ü bir türlü göremezse büyük bir hata olduğunu anlar, işlemi iptal eder ve faks yeniden gönderilir. Bozulmadan dolayı 13. ya da 14. bite gelinmiş ve hâlâ bir sembol bulunamamışsa yine hata anlaşılır ve sayfanın yeniden gönderilmesi istenir.

Sıkıştırmanın başarısı her zaman gönderilen dokümanın içeriğine bağlıdır; doküman ister resim ister metin olsun görüntü olarak gider. Siyah-beyaz resimlerde başarılıdır; gray scale ya da renkli bir resim gönderildiğinde başarı düşer. Hoca, faksla pek renkli gönderim yapılmadığını, en fazla gray scale gönderildiğini ekledi.

6. İki Boyutlu Kodlama: Modified READ

6.1 Neden iki boyutlu kodlama?

Modemlerde olduğu gibi fakslarda da tek boyutlu ve iki boyutlu kodlama vardır. Tek boyutlu kodlama gray scale görüntülerde iyi sonuç vermediği için iki boyutlu kodlama geliştirilmiştir. İki boyutlu kodlama seçimliktir: genelde tek boyutlu kodlanır, iki boyutlu kodlama seçenek olarak bırakılır.

Gönderilen satırın tek mi iki boyutlu mu sıkıştırıldığı bilgisinin karşı tarafa aktarılması gerekir. Bunun için EOL kodundan sonra fazladan bir bit kullanılır:

- Bit 1 ise sonraki satır tek boyutlu kodlanmıştır.
- Bit 0 ise sonraki satır iki boyutlu kodlanmıştır.

Tek boyutlu kodlama Modified Huffman'dı: run değerleri için 0-63 ve 64'ün katları olarak iki tablo (Termination ve Make-up). İki boyutlu kodlama daha karmaşıktır ve başarısı daha yüksektir. Hoca, elle kodlamanın zor olduğunu ama hazır bir programla işin farklı olduğunu söyledi.

İki boyutlu kodlamaya **Modified READ (MR)** denir; READ, Relative Element Address Designate'in kısaltmasıdır. Kendi içinde farklı modlara ayrılır ve belirli kurallar çerçevesinde çalışır.

> **Ek bilgi (derste anlatılmadı):** Altyazıda kısaltma "MMR" olarak da geçiyor. MR (Modified READ) T4'teki (Grup 3) iki boyutlu kodlamanın adıdır; MMR (Modified Modified READ) ise T6'da (Grup 4) kullanılan ve periyodik 1D satır gerektirmeyen sürümüdür.

6.2 Coding line ve reference line

Kodlanacak satıra **coding line** denir. Bu satır tek boyutlu kodlanacaksa run'lar (örneğin 5 siyah, 3 beyaz, 4 siyah, …) sırayla tablodan alınır. İki boyutlu kodlanacaksa hemen üstündeki satıra bakılır; buna **reference line** denir. Yeni satır, üstündeki satır referans alınarak kodlanır.

Dokümanlarda art arda gelen satırlar arasında çoğu zaman çok küçük farklar olur. Örneğin sayfanın başı yalnız beyaz pels'lerden oluşan bir satır olabilir; bu satır tek boyutlu kodlanır. Bir alttaki satırda yalnız bir ya da üç siyah nokta varsa ve geri kalanı beyazsa, iki satır arasındaki değişim çok az olduğundan bu satırı iki boyutlu kodlamak daha iyi sıkıştırma sağlar.

6.3 Kaç satır iki boyutlu kodlanır?

Slayt (ekrandan, 00:40:00):

- 2D kodlama, 1D kodlamadan daha az güvenilirdir. Bir hata oluştuğunda bütün doküman etkilenir. Bu sebepten dolayı belli aralıklarla 2D kodlama kesilir ve bir sonraki satır 1D kodlamayla kodlanır.
- T4 (Grup 3): bir satır 1D kodlamayla kodlandıktan sonra en fazla $k-1$ satır 2D kodlamayla kodlanır (standart çözünürlük $k = 2$, yüksek çözünürlük $k = 4$).
- T6 (Grup 4) kodlamada böyle bir zorunluluk yoktur.

Örneğin T4 standardında, Grup 3'te, yüksek çözünürlükte $k = 4$ ise bir satır 1D kodlandıktan sonra arkasından gelen 3 satır 2D kodlanır, sonra tekrar 1D'ye geçilir. T6'da bütün satırlar 2D kodlanabilir.

2D kodlamada bir hata yapıldığında hata bulunamayabilir ve bütün doküman karşıya hatalı gidebilir. İdeal olan belirli aralıklarla 2D kodlamayı kesip bir satırı 1D kodlamaktır; Grup 3'te bu sağlanmaya çalışılır. Böylece 2D kodlamada hata yapılsa bile en fazla birkaç satır hatalı olur ve sonra düzelir.

Hoca, kullanıcının faks makinesini alıp fişe takıp kullanmaya başladığını, kitapçığı açıp hangi modda (Grup 3, Grup 4) çalıştıracağına karar verebileceğini ama genelde varsayılan ayarla geldiğini söyledi.

7. İki Boyutlu Kodlamanın Üç Modu

İki boyutlu kodlamada, tek boyutludaki Termination ve Make-up kodlarına benzer biçimde üç farklı mod vardır. Slaytta "2D kodlamada 3 farklı mod vardır. 2D kodlama, bu 3 farklı mod arasında değişimli kodlama yapar" yazar (ekrandan, 00:44:50). Hoca bu kuralları elindeki bir kağıttan okuyarak anlattığını, akılda tutmanın zor olduğunu ve ezberlemek gerekmediğini; bir algoritma yazılacaksa bu kuralların zaten verileceğini söyledi.

7.1 Konum değişkenleri

Kodlama satırında a_0, a_1, a_2; referans satırında b_1, b_2 değerleri vardır. Bunlar satırlardaki belirli pels'lerin (run'ların) başlangıçlarını gösterir:

- a_0: kodlama satırında başlangıç beyaz pel'i. İlk taramada, her scanline'ın başına eklenen beyaz pel'i gösterir.
- a_1: a_0'dan sonra gelen ilk siyahın (sonraki run'ın) başlangıcı.

- a_2: ondan sonra gelen beyazın (bir sonraki run'ın) başlangıcı.
- b_1, b_2: referans satırındaki birinci ve ikinci run'ların başlangıcı.

Hoca, satır başına eklenen beyaz pel ile a_0'ın başlangıcını karıştırmamak gerektiğini, a_0'ın ilk başta tam olarak orayı gösterdiğini belirtti.

7.2 Pass mode

Slayt (ekrandan, 00:44:50):

- b_2, a_1'in sol tarafındadır.
- a_1 ve b_1 aynı hizada olamaz.
- Bu mod tanımlandığında Pass Mode (`0001`) + (b_1b_2) uzunluğu yazılır.
- a_0, b_2'nin altına taşınır; b_1 ve b_2 güncellenir; a_1 ve a_2 değişmez.

Algoritma olarak bakılır: b_2, a_1'in solunda mı? Evet. a_1 ile b_1 aynı hizada mı? Hayır. O zaman bu iki satır için pass mode kullanılır. Pass mode'un kendi kodu `0001`'dir; arkasına b_1 ile b_2 arasındaki run uzunluğu gelir. Hoca, örneğin run değeri 2 olan siyah bir pel varsa Termination tablosundan onun kodunun yazılacağını söyledi.

Sonra değişkenler yer değiştirir: a_0, b_2'nin altına taşınır; b_1 bir sonraki run'ı, b_2 ondan sonraki run'ı gösterir; a_1 ve a_2 değişmez. Yeni durumda b_2, a_1'in sağında kalıyorsa artık pass mode değildir; diğer iki moddan biridir.

> **Ders sonrası not:** Slayt ve hoca pass mode'da `0001`'in arkasına b_1b_2 uzunluğunun yazıldığını söyledi. ITU-T T.4'te pass mode kodu büyük olasılıkla yalnız `0001`'dir; b_1b_2 uzunluğu ayrıca gönderilmez, çünkü alıcı referans satırından b_2'nin yerini kendisi bulabilir.

7.3 Vertical mode

Slayt (ekrandan, 00:50:00): a_1 ile b_1 arasında sağa veya sola doğru 2'den fazla "pels" olmamalıdır. Yedi farklı durum için 7 farklı kod üretilir:

```
a1b1 =  0   V(0)   -> 1
a1b1 = -1   VR(1)  -> 011
a1b1 = -2   VR(2)  -> 000011
a1b1 = -3   VR(3)  -> 0000011
a1b1 =  1   VL(1)  -> 010
a1b1 =  2   VL(2)  -> 000010
a1b1 =  3   VL(3)  -> 0000010
```

a_1 ile b_1 alt alta gelmiş olabilir (V0). a_1, b_1'in bir, iki ya da üç pel sağında kalabilir; buna **vertical right** denir. a_1, b_1'in solunda kalıyorsa **vertical left** denir. Bu kodlar önceden çıkarılmıştır.

Hoca "ikiden fazla olmayacak deyip neden tabloda 3 var?" sorusunu kendisi sordu. Daha sonra örnek sırasında (00:57:09) slayta yanlışlıkla "ikiden" yazdığını, bunun "üçten fazla olmayacak" olması gerektiğini söyledi. Doğru koşul $|a_1b_1| \leq 3$'tür; tablodaki yedi durum da bunu gösterir.

Slayttaki şekillerde (b1) a_1, b_1'in solunda ve (b2) a_1, b_1'in sağında gösterilir; "Run length a_1b_1 coded. New a_0 becomes old a_1." Hoca örnekte a_1'in b_1'in 3 birim solunda olduğu durumu VL(3) olarak, sağında olduğu durumu VR olarak gösterdi.

Kodlamadan sonra yer değiştirme: a_0, a_1'in yerine; a_1, a_2'nin yerine; a_2 bir sonraki run'ın başına geçer. Referansta b_1, b_2'nin yerine; b_2 bir sonraki run'ın başına geçer. Sonra yine hangi moda girileceğine bakılır.

7.4 Horizontal mode

Slayt (ekrandan, 00:52:10):

- a_1 ile b_1 arasında sağa veya sola doğru 3'ten fazla "pels" olmalıdır.
- Horizontal mode'un kodu `001` ve a_0a_1 ile a_1a_2 arası pels'in MH'ye göre kod değeri gönderilir.
- a_0, a_2'nin yerine geçer; a_1, a_2, b_1 ve b_2 güncellenir.

- Şekil notu: “New a_0 becomes old a_2. Run lengths a_0a_1 (white) and a_1a_2 (black) coded.”

Slayttaki (c1) örneğinde b_1 ile a_1 arasında sağa doğru 3’ten fazla pel vardır; o halde horizontal mode’dur. Kod `001`’dir. Sonra Modified Huffman’a göre a_0a_1 arası sayılır: 3 beyaz; tablodan 3 beyazın kodu alınır. a_1a_2 arası 7 siyahtır; tablodan 7 siyahın kodu alınır. Sonra güncellemeler yapılır.

7.5 Toparlama

Hoca üç modu şöyle özetledi: Tek boyutlu kodlama çok kolaydır; tabloya bakılır ve kodlar alınır. İki boyutlu kodlamada doküman tamamen 2D kodlanabilir ama hataya çok açıktır. Grup 3’te $k-1$ satır 2D kodlanır ve bu şekilde değiş tokuş yapılır. Kodlanacak satırın üstündeki satır referans alınır ve üç moddan hangisine uyduğuna bakılır:

- Pass ise: pass mode’un kodu + b_1b_2 uzunluğu.
- Horizontal ise: horizontal’ın kodu + a_0a_1 ve a_1a_2 mesafelerinin MH kodu.
- Vertical ise: a_1 ve b_1’in konumuna göre ilgili kod.

Ayrıca bir **extension** kodu vardır: Gönderim iptal edilmek istendiğinde ya da bir hata olduğu anlaşıldığında bu kod gönderilerek işlem sonlandırılır.

Hoca öğrencilerden, örnekleri takip ederken kolaylık olsun diye kuralları gösteren slaytın fotoğrafını çekmelerini istedi.

8. İki Boyutlu Kodlama Örneği

Hoca bir satırın iki boyutlu kodlanmasını slaytlardaki “Short example” ile adım adım gösterdi (ekrandan, 00:58:10 ve 01:00:00). Her adımda önce pass, sonra vertical, sonra horizontal kontrol edilir.

Adım 1. Kodlama satırında a_0, a_1, a_2; referans satırında b_1, b_2 verilmiştir. Pass mode mu? b_2, a_1’in solunda değil; pass değil. Vertical mi? a_1 ile b_1 arasında 3’ten fazla pel yok. a_1, b_1’in bir pel solunda; o halde VL(1). Slayt: “$a_1b_1 = 1$, Vertical Mode” ve çıktı `010`.

Yer değiştirme: a_0 eski a_1’in yerine, a_1 eski a_2’nin yerine, a_2 bir sonraki run’ın başına gelir; b_1, b_2 de güncellenir.

Adım 2. Pass mi? b_2, a_1’in solunda değil. Vertical mi? a_1 ile b_1 üst üste gelmiş; V(0). Kodu `1`. Çıktı: `010 1`.

Adım 3. Pass mi? b_2, a_1’in solunda değil. Vertical mi? a_1 ile b_1 arasındaki pel sayısı 3’ten fazla; o halde horizontal mode. Slayt: “$|a_1b_1| > 3$, Horizontal Mode, 3 White, 4 Black” ve çıktı:

```
010  1  001 + 1000 + 011
```

Horizontal’ın kodu `001`, 3 beyazın Termination kodu `1000`, 4 siyahın Termination kodu `011`’dir (tablodan doğrulandı). Hoca altyazıda önce 4 siyah, sonra 3 beyaz diye saydı; slayttaki sıra 3 beyaz, 4 siyahtır (a_0a_1 beyaz, a_1a_2 siyah).

Adım 4. b_2, a_1’in solunda; pass mode. Pass mode’un kodu yazılır, arkasına b_1 ile b_2 arasındaki run değeri (2 beyaz) Termination tablosundan eklenir.

Adım 5. b_2, a_1’in solunda değil; pass değil. a_1 ile b_1 arasında 3’ten fazla pel yok; vertical. a_1, b_1’in sağında ve aradaki fark 2; VR(2), kodu `000011`.

Adım 6. Yine vertical mode: a_1, b_1’in solunda ve arada iki pel var; VL(2), kodu `000010`.

Adım 7. Bir horizontal daha gelir ve satır biter. Satırın sonuna EOL kodu konur.

Satır bitince sonraki satıra geçilir. $k-1$ tane 2D satır kodlanmışsa sonraki satır tek boyutlu kodlanır. Kodlanmamışsa bu sefer biten satır reference line, alttaki satır coding line olur ve değişkenler yerlerine konarak kodlamaya devam edilir.

Hoca bunun anlatırken uzun sürdüğünü ama aslında zahmetli olmadığını söyledi: Karşımızda iki dizi vardır; siyahlar 0, beyazlar 1 olsun. a’lar ve b’ler başlangıç yerleridir. Yapılan iş `if` kontrolleriyle output stream’e çıktıyı yerleştirmektir. Böylece modemlerde ve fakslarda ne tür sıkıştırmaların kullanıldığı görülmüş oldu.

9. Aritmetik Kodlamaya Giriş

Aradan sonra hoca istatistiksel yöntemlere devam etti ve "çok enteresan ve değişik" dediği **aritmetik kodlamayı** (arithmetic coding) anlattı.

Şimdiye kadar görülen yöntemlerde sembollere kodlar atanıyor ve arka arkaya gelen karakterlerin kodları eklenerek sıkıştırma yapılıyordu. Aritmetik kodlamada sembollere kod verilmez; bütün dokümana tek bir kod verilir. Bu kod 0 ile 1 arasında bir reel sayıdır.

Hoca önce bir hatırlatma yaptı: İlk olarak Shannon-Fano görülmüştü; olasılıklar yaklaşık ortalanacak şekilde ikiye bölünüyordu ve olasılıklar 2'nin negatif kuvvetleri olduğunda çok iyi sonuç veriyordu. Sonra Huffman görüldü: olasılık ya da frekans değerleriyle bir ağaç kuruluyor, prefix kodlar elde ediliyor ve ağaç yukarıdan aşağı okunarak kodlar bulunuyordu. Huffman ağacının bütün prefix kodları üretmediği ve aradaki prefix kodlar da kullanılırsa header'ın küçüleceği fikriyle canonical Huffman anlatılmıştı. Hoca ayrıca geçen derste emin olamadığı tavan fonksiyonu konusunu kontrol ettiğini ve söylediğinin doğru olduğunu belirtti.

Slayt (ekrandan, 01:20:00):

- Sembollerin olasılık değerleri ikinin negatif kuvvetlerine sahipse Shannon-Fano iyi sonuç veriyordu.
- Huffman yöntemi, Shannon-Fano yönteminden daha iyi sonuç veriyordu.
- Bir sembolün olasılığı 0,4 ise bu sembole atanan bit $-\log_2 0{,}4 \approx 1{,}32$ (1 veya 2 bit).
- Aritmetik kodlamada tek bir sembole değil, bütün input stream'e bir kod atanır.
- Atanan bu kod değeri $[0, 1)$ aralığında bir sayı olur.
- Eğer kod "9746509" ise bu "0,9746509" olarak yorumlanmalıdır. Ancak "0" output stream'e yazılmaz.
- İki pass'te çalışan bir yöntemdir. İlk pass'te sembollerin kullanım sıklıkları çıkarılır, ikinci pass'te sıkıştırma işlemi gerçekleştirilir.

Aralık 0'da kapalı, 1'de açıktır: dokümanın kodu belki 0 olabilir ama hiçbir zaman 1 olamaz. Aritmetik kodlamanın bir ondalık sayı üreteceği bilindiği için baştaki "0," hiç yokmuş gibi davranılır.

Hoca aritmetik kodlamanın Huffman'a yakın bir başarı verdiğini ama her zaman Huffman'dan "bir tık daha iyi" olduğunu söyledi.

10. Aralık Daraltma ve İlk Örnek

10.1 Sembolleri [0, 1) aralığına dağıtmak

Üç sembol olsun: a_1, a_2, a_3; olasılıkları $0{,}4$, $0{,}5$, $0{,}1$. Bu semboller olasılıklarına göre $[0, 1)$ aralığına dağıtılır:

- a_1: $[0; 0{,}4)$
- a_2: 0,4'ten başlar, olasılığı 0,5 eklenir: $[0{,}4; 0{,}9)$
- a_3: 0,9'dan başlar, 0,1 eklenir: $[0{,}9; 1)$

Sembol sayısı 3, 10 ya da 150 olsun, olasılıkların toplamı 1'dir ve her sembol olasılığı kadar bir alt aralık alır. Her sembolün bir başlangıç ve bir bitiş değeri vardır.

10.2 Yeni low ve high hesabı

Amaç şudur: Bir sembol okunduğunda aralık daralır, ikinci sembol geldiğinde biraz daha daralır ve her yeni sembolle sayı daha da kesinleşir. Başlangıçta low 0, high 1'dir. Her sembolde yeni low ve high hesaplanır (ekrandan, 01:30:00):

$$\text{NewHigh} = \text{OldLow} + \text{Range} \times \text{HighRange}(X)$$
$$\text{NewLow} = \text{OldLow} + \text{Range} \times \text{LowRange}(X)$$

Burada $\text{Range} = \text{OldHigh} - \text{OldLow}$, X okunan semboldür; $\text{HighRange}(X)$ ve $\text{LowRange}(X)$ o sembolün aralığının üst ve alt sınırlarıdır.

10.3 Örnek: a2a2a2a3

Kodlanacak string üç tane a_2 ve bir a_3'ten oluşuyor: `a2a2a2a3` (ekrandan, 01:30:00).

Birinci a_2: Başlangıç aralığı $[0, 1)$. Eski low 0, range $1 - 0$.

$$\text{Low} = 0 + (1 - 0) \times 0{,}4 = 0{,}4 \qquad \text{High} = 0 + (1 - 0) \times 0{,}9 = 0{,}9$$

Dokümanda şu an bir sembol var ve aralık $[0{,}4; 0{,}9)$'a daraldı.

İkinci a_2: Eski low 0,4, range $0{,}9 - 0{,}4 = 0{,}5$.

$$\text{Low} = 0{,}4 + 0{,}5 \times 0{,}4 = 0{,}6 \qquad \text{High} = 0{,}4 + 0{,}5 \times 0{,}9 = 0{,}85$$

Üçüncü a_2: Eski low 0,6, range $0{,}85 - 0{,}6 = 0{,}25$.

$$\text{Low} = 0{,}6 + 0{,}25 \times 0{,}4 = 0{,}7 \qquad \text{High} = 0{,}6 + 0{,}25 \times 0{,}9 = 0{,}825$$

a_3: Eski low 0,7, range $0{,}825 - 0{,}7 = 0{,}125$.

$$\text{Low} = 0{,}7 + 0{,}125 \times 0{,}9 = 0{,}8125 \qquad \text{High} = 0{,}7 + 0{,}125 \times 1 = 0{,}825$$

Sonuç aralığı $[0{,}8125; 0{,}825)$'tir (python ile doğrulandı). Bu dokümanı kodlamak için bu iki sayı arasındaki bir değer ya da yalnız low değeri (noktadan sonrası, `8125`) kullanılır. Hoca "Ne kadar enteresan değil mi? Bu sayıdan nasıl `a2a2a2a3`'ü açacağım?" diye sordu; aritmetik kodlamayı ilginç yapan şey budur.

Dokümanda yüzlerce karakter olabilir; her yeni karakterle aralık küçülür. 0 ile 1 arasında noktadan sonra sonsuz uzunlukta sayı vardır; aralık içindeki bir değer ya da low değerinin noktadan sonrası dokümanın kodlanmış hali olarak alınır.

10.4 Kurallar ve ortalama kod uzunluğu

Kurallar (slayttaki maddeler, altyazıdan):

- İşleme $[0, 1)$ aralığıyla başlanır.
- Input stream'den gelen her S sembolü için şu adımlar tekrarlanır: Geçerli aralık alt aralıklara bölünür; alt aralıkların uzunluğu sembollerin olasılıklarıyla orantılıdır. S için bir alt aralık seçilir ve yeni geçerli aralık olarak tanımlanır.
- Bütün input stream işlendiğinde çıkış, geçerli aralıkta kalan herhangi bir sayıdır.
- Her sembol işlendikçe geçerli aralık küçülür ve bu, kodun daha fazla bitle ifade edilmesine sebep olur.
- Sonuç tek bir sayıdır. Bu sayı tek bir sembolü değil, bütün input stream'i temsil eder.

Ortalama kod uzunluğu: Huffman'da redundancy hesaplanırken $H - S$ kullanılıyordu; H entropi, S ortalama mesaj uzunluğuydu ve $S = \sum P_i l_i$ (olasılık çarpı kaç bitte kodlandığı) ile bulunuyordu. Aritmetik kodlamada ise:

$$\text{Ortalama kod uzunluğu} = \frac{\text{output'un bit uzunluğu}}{\text{input stream'deki sembol sayısı}}$$

Önceki örnekte input'ta 4 sembol vardı; `8125` sayısı kaç bitte kodlanıyorsa o bit uzunluğu 4'e bölünür.

10.5 Header

Karşı tarafın kodu açabilmesi için bazı bilgilere ihtiyacı vardır. Sıkıştırılmış veri output stream'e yazılmadan önce semboller ve onların sıklık değerleri çıkışa (header'a) yazılır. Hoca, Huffman'da da bunun olduğunu ama orada ağacı göndermeyi tercih ettiklerini hatırlattı. Karşı taraf bu bilgilerle ilgili tabloyu oluşturur ve ters adımları yaparak orijinal metne ulaşır. Alt ve üst sınırlar low ve high değişkenleriyle tutulur; başlangıçta low 0, high 1'dir.

11. SWISS MISS Örneği: Kodlama ve Kod Çözme

11.1 Tablonun kurulması

Input `SWISS MISS` olsun (ortadaki karakter boşluk). S 5 kez, W 1, I 2, M 1, boşluk 1 kez geçer; toplam 10 sembol. Tablo (ekrandan, 01:40:00):

```
Char   Freq  Prob.          Range        CumFreq
                          Total CumFreq =   10
S       5    5/10 = 0.5   [0.5, 1.0)        5
W       1    1/10 = 0.1   [0.4, 0.5)        4
I       2    2/10 = 0.2   [0.2, 0.4)        2
M       1    1/10 = 0.1   [0.1, 0.2)        1
boşluk  1    1/10 = 0.1   [0.0, 0.1)        0
```

Semboller aşağıdan yukarı yerleştirilir: 0 ile başlanır, boşluğun olasılığı (0,1) eklenir: 0,1. 0,1'e M'nin olasılığı eklenir: 0,2. 0,2'ye I eklenir: 0,4. 0,4'e W eklenir: 0,5. 0,5'e S eklenir: 1. S'nin alt sınırı 0,5, üst sınırı 1'dir.

> **Hocanın vurgusu:** Kümülatif frekans önemli bir kavramdır; kod çözme işleminde de kullanılır.

Kümülatif frekans sırayla eklenerek bulunur: boşluk 0; boşluktan 1 tane var, 0'a eklenir, M için 1; M'den 1 tane, I için 2; I'dan 2 tane, W için 4; W'dan 1 tane, S için 5; S'den 5 tane, toplam 10.

11.2 Kodlama

Kodlama tablosu (ekrandan, 01:40:00):

```
S  L  0.0 + (1.0 - 0.0) x 0.5 = 0.5
   H  0.0 + (1.0 - 0.0) x 1.0 = 1.0
W  L  0.5 + (1.0 - 0.5) x 0.4 = 0.70
   H  0.5 + (1.0 - 0.5) x 0.5 = 0.75
I  L  0.7 + (0.75 - 0.70) x 0.2 = 0.71
   H  0.7 + (0.75 - 0.70) x 0.4 = 0.72
S  L  0.71 + (0.72 - 0.71) x 0.5 = 0.715
   H  0.71 + (0.72 - 0.71) x 1.0 = 0.72
S  L  0.715 + (0.72 - 0.715) x 0.5 = 0.7175
   H  0.715 + (0.72 - 0.715) x 1.0 = 0.72
_  L  0.7175 + (0.72 - 0.7175) x 0.0 = 0.7175
   H  0.7175 + (0.72 - 0.7175) x 0.1 = 0.71775
M  L  0.7175 + (0.71775 - 0.7175) x 0.1 = 0.717525
   H  0.7175 + (0.71775 - 0.7175) x 0.2 = 0.717550
I  L  0.717525 + (0.71755 - 0.717525) x 0.2 = 0.717530
   H  0.717525 + (0.71755 - 0.717525) x 0.4 = 0.717535
S  L  0.717530 + (0.717535 - 0.717530) x 0.5 = 0.7175325
   H  0.717530 + (0.717535 - 0.717530) x 1.0 = 0.717535
S  L  0.7175325 + (0.717535 - 0.7175325) x 0.5 = 0.71753375
   H  0.7175325 + (0.717535 - 0.7175325) x 1.0 = 0.717535
```

Son low değeri $0{,}71753375$'tir (python ile doğrulandı). Bu sayı ya da onun ikili karşılığı `SWISS MISS`'in sıkıştırılmış halidir.

11.3 Kod çözme

Kod çözücü karşı yönde çalışır (ekrandan, 01:43:30):

- Kod çözücü, sembol ve onların aralık değerlerini okuyarak başlangıçta oluşturulmuş olan tabloyu tekrar oluşturur (char, freq, prob, range, cumfreq). Sıklık ya da olasılık değerleri header'da bildirildiği için bu mümkündür.
- İlk dijit olarak 7'yi okur ve bilir ki bu değer 0,7'dir.

Her adımda kodun hangi aralıkta olduğuna bakılır, o sembol çıkışa yazılır, kodtan sembolün low değeri çıkarılır ve sonuç sembolün range'ine bölünür:

```
Char  Code - low                                  Range
S     0.71753375 - 0.5 = 0.21753375  /0.5  =  0.4350675
W     0.4350675  - 0.4 = 0.0350675   /0.1  =  0.350675
I     0.350675   - 0.2 = 0.150675    /0.2  =  0.753375
S     0.753375   - 0.5 = 0.253375    /0.5  =  0.50675
S     0.50675    - 0.5 = 0.00675     /0.5  =  0.0135
_     0.0135     - 0   = 0.0135      /0.1  =  0.135
M     0.135      - 0.1 = 0.035       /0.1  =  0.35
I     0.35       - 0.2 = 0.15        /0.2  =  0.75
S     0.75       - 0.5 = 0.25        /0.5  =  0.5
S     0.5        - 0.5 = 0           /0.5  =  0
```

- 0,7175... değeri $[0{,}5; 1)$ aralığındadır; bu S'nin aralığıdır, çıkışa S yazılır. S'nin low değeri 0,5 çıkarılır ve S'nin range'i 0,5'e bölünür: 0,435...
- 0,435 $[0{,}4; 0{,}5)$ aralığında, W. W'nun low'u 0,4 çıkarılır, range'i 0,1'e bölünür: 0,35...
- 0,35 I'nın aralığında. Böyle devam edilir: 0,75 → S; 0,50 → S (aralık 0,5'te kapalıdır, bu yüzden S); 0,01 → boşluk; 0,135 → M; 0,35 → I; 0,75 → S; 0,5 → S.
- Değer 0'a gelince çözülmesi gereken veri bitmiş demektir. Sonuç `SWISS MISS`'tir (python ile doğrulandı).

Hoca bu algoritmayla ilk karşılaştığında gerçekten hayranlık duyduğunu, tek bir sayıdan koskoca bir metnin geri bulunmasının çok şık olduğunu söyledi.

12. Son Sembol Problemi ve EOF

Hoca öğrencilere buraya kadar soru olup olmadığını sordu, sessizliği "yok" diye kabul edip devam etti. Bazen beklenmedik durumlarla karşılaşılabilir.

Üç sembol ve olasılıkları (ekrandan, 01:50:00):

```
Char   Prob.      Range
a1     0.001838   [0.998162, 1.0)
a2     0.975      [0.023162, 0.998162)
a3     0.023162   [0.0, 0.023162)
```

Kodlanacak string `a2a2a1a3a3`'tür. Kodlama:

```
a2   0.0 + (1.0 - 0.0) x 0.023162 = 0.023162
     0.0 + (1.0 - 0.0) x 0.998162 = 0.998162
a2   0.023162 + .975 x 0.023162 = 0.04574495
     0.023162 + .975 x 0.998162 = 0.99636995
a1   0.04574495 + 0.950625 x 0.998162 = 0.99462270125
     0.04574495 + 0.950625 x 1.0 = 0.99636995
a3   0.99462270125 + 0.00174724875 x 0.0 = 0.99462270125
     0.99462270125 + 0.00174724875 x 0.023162 = 0.99466317...
a3   0.99462270125 + 0.0000404697... x 0.0 = 0.99462270125
     0.99462270125 + 0.0000404697... x 0.023162 = 0.99462363...
```

Output stream'e yazılan sayı $0{,}99462270125$'tir; buraya kadar sorun yoktur (python ile doğrulandı).

Kod çözme:

```
Char  Code - low                                    Range
a2    0.99462270125 - 0.023162 = 0.97146070125 /0.975   = 0.99636995
a2    0.99636995 - 0.023162   = 0.97320795      /0.975   = 0.998162
a1    0.998162 - 0.998162     = 0.0             /0.00138 = 0.0
a3    0.0 - 0.0               = 0.0             /0.023162 = 0.0
a3    0.0 - 0.0               = 0.0             /0.023162 = 0.0
```

a_1'e gelindiğinde değer birden sıfırlanır. Sıfıra gelince durulduğu için input `a2a2a1` olarak çözülür; a_3'ler kaybolur. Hoca bunun algoritmanın bir "bug"ı olduğunu ve denemelerle ortaya

çıktığını söyledi: Input stream'in en son sembolünün aralığının alt değeri 0 ise bu problem yaşanır. Son sembol a_2 ya da a_1 olsaydı bu problem yaşanmazdı.

Çözüm: Dokümanın sonuna bir **EOF (end of file)** sembolü eklenir. Bu da bir sembol gibi düşünülür ve olasılığı çok çok küçük tutulur. Slayt (ekrandan, 01:51:20):

```
Char   Prob.      Range
eof    0.000001   [0.999999, 1.0)
a1     0.001837   [0.998162, 0.999999)
a2     0.975      [0.023162, 0.998162)
a3     0.023162   [0.0, 0.023162)
```

String `a3a3a3a3eof` kodlanınca:

```
a3   0.0 + (1.0 - 0.0) x 0.0 = 0.0
     0.0 + (1.0 - 0.0) x 0.023162 = 0.023162
a3   0.0 + .023162 x 0.0 = 0.0
     0.0 + .023162 x 0.023162 = 0.000536478244
a3   0.0 + 0.000536478244 x 0.0 = 0.0
     0.0 + 0.000536478244 x 0.023162 = 0.000012425909087528
a3   0.0 + 0.000012425909087528 x 0.0 = 0.0
     0.0 + 0.000012425909087528 x 0.023162 = 0.000000287808906285323 5
eof  0.0 + 0.000000287808906285323 5 x 0.999999 = 0.000000287808618476417 2
     0.0 + 0.000000287808906285323 5 x 1.0 = 0.000000287808906285323 5
```

Son sembol artık a_3 değil eof olduğu için a_3'ün alt sınırının 0 olmasının bir önemi kalmaz; a_3'ler rahatlıkla çözülür ve eof'a gelindiğinde doküman bitmiş olur. Slayttaki not: "Ek bir EOF sembolü kullanmak gerekir veya kodlayıcı output stream'e önce input stream'in boyutunu yazar ve kontrolü böyle sağlar."

Altyazıda hoca bu sembolü "end of line" diye de söyledi; slayttaki sembol eof'tur. Önemli olan, alt sınırı 0 olan sembolün dokümanın sonunda olmamasıdır; bu sembol a_3 değil a_2 ya da a_1 de olabilir.

13. Tamsayılarla Aritmetik Kodlama

13.1 Neden tamsayı?

Bu işlemler kolaydır ama pratik değildir, çünkü çok fazla ondalıklı (float) sayıyla uğraşılır. Bu yüzden aritmetik kodlama, hep tamsayılarla işlem yapılacak şekilde değiştirilmiştir. Hoca bu sürümü kendisine göre daha karmaşık bulduğunu söyledi.

Hoca ayrıca şunu söyledi: Örneğin 1000 MB'lık bir dosya tek bir sayıyla kodlanır ama o tek sayıyı ifade etmek için 500 KB gerekebilir. Metnin içeriği de çok önemlidir.

13.2 Ölçekleme ve öteleme

High ve low değişkenleri 4 dijit uzunluğundadır. En soldaki dijite bakılarak ötelemeler yapılır: low değişkenine sağdan 0, high değişkenine sağdan 9 girilerek ötelenir.

Slayt (ekrandan, 02:00:00), "SWISS MISS stringinin kodlama işlemini gösterelim":

- 1. kolon okunacak input stream.
- 2. kolon LOW ve HIGH yeni değerleri.
- 3. kolon bu değerlerin ölçeklendirilmiş hali (LOW ve HIGH değerleri 10 bin ile çarpılır, HIGH değerinden 1 çıkarılır).
- 4. kolon output stream'e gönderilecek dijit.
- 5. kolon sola ötelemeden sonraki yeni LOW ve HIGH değerleri.

```
     2                                  3     4    5
S  L = 0+(1-0)x0.5 = 0.5               5000        5000
   H = 0+(1-0)x1.0 = 1.0               9999        9999
W  L = 0.5+(1-.5)x0.4 = 0.7            7000   7    0000
   H = 0.5+(1-.5)x0.5 = 0.75           7499   7    4999
I  L = 0+(0.5-0)x0.2 = 0.1             1000   1    0000
```

```
   H = 0+(0.5-0)x0.4 = 0.2             1999   1    9999
S  L = 0+(1-0)x0.5 = 0.5               5000        5000
   H = 0+(1-0)x1.0 = 1.0               9999        9999
S  L = 0.5+(1-0.5)x0.5 = 0.75          7500        7500
   H = 0.5+(1-0.5)x1.0 = 1.0           9999        9999
_  L = 0.75+(1-0.75)x0.0 = 0.75        7500   7    5000
   H = 0.75+(1-0.75)x0.1 = 0.775       7749   7    7499
M  L = 0.5+(0.75-0.5)x0.1 = 0.525      5250   5    2500
   H = 0.5+(0.75-0.5)x0.2 = 0.55       5499   5    4999
I  L = 0.25+(0.5-0.25)x0.2 = 0.3       3000   3    0000
   H = 0.25+(0.5-0.25)x0.4 = 0.35      3499   3    4999
S  L = 0+(0.5-0)x0.5 = .25             2500        2500
   H = 0+(0.5-0)x1.0 = 0.5             4999        4999
S  L = 0.25+(0.5-0.25)x0.5 = 0.375     3750        3750
   H = 0.25+(0.5-0.25)x1.0 = 0.5       4999        4999
```

Adımlar:

- **S:** 0,5 ve 1 değerleri 10.000 ile çarpılır: 5000 ve 10.000; high'dan 1 çıkarılır: 9999. Sol dijitler (5 ve 9) aynı değil; değerler olduğu gibi alınır.
- **W:** Aralık $[0{,}5; 1)$. W'nun sınırları 0,4 ve 0,5; yeni low 0,7, high 0,75. 10.000 ile çarpılınca 7000 ve 7500; 1 çıkarılınca 7499. En soldaki dijitler aynı (7); bu dijit output stream'e yazılır. Low'da kalan 000'a sağdan 0 ötelenir: 0000. High'da kalan 499'a sağdan 9 ötelenir: 4999.
- **I:** Artık aralık $[0; 0{,}5)$'tir. Low $0 + 0{,}5 \times 0{,}2 = 0{,}1$, high $0 + 0{,}5 \times 0{,}4 = 0{,}2$. Ölçekli 1000 ve 1999; sol dijitler aynı (1), çıkışa yazılır; low 0000, high 9999 olur.
- **S:** Aralık yine $[0; 1)$ (9999 aslında 1 olarak alınır). 5000 ve 9999; dijitler farklı.
- **S:** Aralık $[0{,}5; 1)$; 7500 ve 9999; dijitler farklı.
- **Boşluk:** Aralık $[0{,}75; 1)$; low 0,75, high 0,775 → 7500 ve 7749. Sol dijit 7 çıkışa yazılır; low 5000, high 7499.
- **M:** Aralık $[0{,}5; 0{,}75)$; 5250 ve 5499. 5 çıkışa yazılır; 2500 ve 4999.
- **I:** Aralık $[0{,}25; 0{,}5)$; 3000 ve 3499. 3 çıkışa yazılır; 0000 ve 4999.
- **S:** Aralık $[0; 0{,}5)$; 2500 ve 4999; farklı.
- **S:** Aralık $[0{,}25; 0{,}5)$; 3750 ve 4999. Sonda low'daki 3750 de çıkışa eklenir.

Çıkışa giden dijitler sırasıyla 7, 1, 7, 5, 3 ve sonda low değeri 3750'dir. Slayt: "Output stream'e «717533750» değeri gönderilir." Bu, ondalık sürümde bulunan $0{,}71753375$ ile aynı sayıdır.

13.3 Tamsayı sürümünde kod çözme

Kod çözmede low ve high'da kümülatif frekanslar kullanılır. Formüldeki "10", kümülatif frekans toplamıdır, yani elimizdeki sembol sayısı. Başlangıçta low 0, high 9999'dur. Elimizdeki kodun ilk 4 dijiti alınır ve bir indeks hesaplanır. İndeks formülünde code, low, high ve kümülatif frekans değerleri yer alır.

> **Ek bilgi (derste anlatılmadı):** Kaynak kitaptaki (Salomon) indeks formülü büyük olasılıkla $\text{index} = \lfloor((\text{Code} - \text{Low} + 1) \times 10 - 1)/(\text{High} - \text{Low} + 1)\rfloor$ biçimindedir. Bulunan indeks, CumFreq tablosunda hangi sembolün aralığına düşüyorsa o sembol çözülür; ardından low ve high o sembolün kümülatif frekanslarıyla güncellenir.

Hoca adımları şöyle anlattı: İlk 4 dijitle indeks hesaplanır, çıkan değer 0,71 gibi düşünülür; 0,71 S'nin aralığındadır, S seçilir. S'ye göre low ve high yeniden hesaplanır. Yeni bir indeks hesaplanır; bu W'nun aralığına düşer. Low ve high'ların sol dijitleri aynıysa koddan bu dijit (7) atılır ve sağdan kodun bir sonraki dijiti eklenir. Sonraki indeks 0,35 olur ve I'ya karşılık gelir. Bu şekilde devam edilir ve sonunda bütün sembollerin çözüldüğü görülür.

> **Hocanın vurgusu:** Hoca bu sürümün sözlü anlatılmasının çok zor olduğunu, öğrencilerin slaytların üzerinden kendilerinin geçmesi gerektiğini söyledi. Özetle: indeksteki değer tabloda hangi olasılık aralığına denk geliyorsa o sembol alınır; low ve high'ın ilk dijitleri aynıysa kodun en soldaki dijiti atılır, sağına yeni bir dijit eklenir ve yeni kodla yeniden indeks hesaplanır.

Hoca, 10 sembolün normalde 10 bayt ile ifade edileceğini, bu yöntemle elde edilen low değeriyle 4 bayta sıkıştırılabileceğini söyledi.

14. PPM: İçerik Tabanlı Sıkıştırmaya Giriş

14.1 Tanım

İkinci aradan sonra hoca, yine istatistik bilgisini kullanan ama "dünyamızın yine değiştiği" farklı türde bir algoritmaya geçti: **PPM (Prediction with Partial String Matching)**.

Slayt (ekrandan, 02:10:00):

- J. Cleary, I. Witten tarafından ortaya atılmış, A. Moffat tarafından da daha iyi hale getirilmiştir.
- PPM, "order-N" Markov modelini kullanan içerik tabanlı bir sıkıştırma yöntemidir.
- Model, sembole bir olasılık değeri atamadan evvel sembolün içeriğini inceler.
- Kodlayıcı ve kod çözücü, text'in ilerisine erişemediğinden her ikisi de geçmişteki text'in içeriğiyle ilgilenir.
- Pratikte bir sembolün içeriği kendisinden önce gelen N adet semboldür.

PPM'in B, C gibi farklı versiyonları vardır ama aralarındaki değişiklikler o kadar küçüktür ki çok önemli değildir.

14.2 Markov modeli hatırlatması

Hoca Markov modelini ikinci haftaya dönerek hatırlattı: Siyah ve beyaz iki piksel olsun. Şu an beyaz bir pikseldeysek bir sonrakinin siyah mı beyaz mı olacağını bilemeyiz; iki durum vardır. Ama bir sonrakinin ne olacağını, şu anki pikselin öncesindeki durum gibi geçmiş bilgisini de kullanarak tahmin ediyorsak buna Markov model denir. "Order" sıradır: geçmişteki kaç sembolün (1, 2, 3, ...) kullanılacağını söyler; adı bu yüzden k-order'dır.

Örneğin bir metinde T harfine olasılık atanacaksa, içerik tabanlı yöntemlerde "T şu kadar geçiyor, toplam şu kadar sembol var, oranla" denmez. Order modeline göre bakılır: Geçmişinde S olan bir T'nin olasılığı, geçmişinde R olan bir T'ninkinden ve geçmişinde C olan bir T'ninkinden farklıdır. Geçmiş tek karakter de olmayabilir: iki karakter kullanan bir modelde T'nin geçmişinde "boşluk S" varken, "AR" varken ve "IC" varken olasılıkları farklıdır.

Hoca slayttaki "içerik tabanlı" ifadesinin aslında "Markov modelini kullanan içerik tabanlı" olarak düşünülmesi gerektiğini ve "pratikte" cümlesinde de bir hata olduğunu belirtti (bkz. 16.1).

14.3 İstatistiksel, statik ve adaptif yöntemler

İstatistiksel model deyince akla gelen şudur: Q kaç kez geçiyor, toplam kaç sembol var, oranlanır ve Q'nun olasılığı bulunur. Bunun içerik tabanlı bir yöntemle ilişkisi yoktur.

Slayt (ekrandan, 02:20:00), **Statik İçerik Tabanlı Yöntemler**:

- Her zaman aynı olasılık değerleri kullanılır. Alfabedeki mümkün olan bütün di-gram ve tri-gram olasılık değerlerinden statik bir tablo oluşturulur.
- Bir S sembolü okunduğunda, kendisinden önce okunmuş C context'ine bağlı olarak tablodan olasılık değerleri çekilerek kullanılır.

Statik yöntemde tabloyu önceden birileri belirli dokümanlar üzerinde hazırlar: "K'dan önce B, A gelmişse olasılığı şu kadardır" gibi. Sembolün olasılığı geçmişine bağlıdır ama sabittir ve hep aynı tablo kullanılır.

Adaptif yöntemlerde önceden hazırlanmış bir tablo yoktur; metin işlendikçe olasılıklar değişir. Hoca bunu dinamik Huffman'a benzetti: Orada her yeni sembol geldiğinde olasılıklar değişiyor ve dokümanın sonuna kadar bir sembole atanan kod da değişebiliyordu. Bu derste PPM adaptif olarak görülecektir.

15. Lipogram ve Doğal Dil Dışı Metinler

Hoca kitapta geçen bir bilgiyi de aktardı: **Lipogram**, belirli bir kural çerçevesinde yazılmış yazılardır. Hoca, Ernest Vincent adlı bir yazarın eserinden söz etti: Bölümlerde A'dan Z'ye kadar harflerle sırayla başlayan, 26. bölümde bütün harfleri kullanan, 27. bölümden itibaren yine birer harf çıkararak 52. bölümde yalnız A ile başlayan kelimelerle biten anlamlı bir metin.

Hoca bunun sıkıştırmayla ilgisini şöyle açıkladı: Lipogram gibi metinlerde sıkıştırma algoritmalarından beklenen sonuçlar alınamaz, çünkü insanlar doğal dilin akışı dışında, kendilerini zorlayarak yazar. Örneğin kelimelerin hepsi A ile başlıyorsa A harflerinin sayısı zorla artırılmıştır; böyle bir metne Huffman uygulandığında normal bir metindeki başarı elde edilmez.

Hoca ayrıca doğada bir kanun olduğundan söz etti: Bir bölgedeki dağların yükseklikleri sıralandığında, küçük sapmalarla, ikincisi birincinin belirli bir katı kadar az, üçüncüsü aynı katta daha az olacak şekilde azalarak gider; doğal dildeki dağılımlar da böyledir.

> **Ders sonrası not:** Hocanın anlattığı 52 bölümlük alfabetik yapı büyük olasılıkla Walter Abish'in "Alphabetical Africa" romanına aittir. Ernest Vincent Wright'ın bilinen lipogramı "Gadsby"dir ve büyük olasılıkla hiç "e" harfi kullanmadan yazılmıştır.

16. PPM'de Order Sayısı ve Context Tablosunun Kurulması

16.1 N neden küçük seçilir?

> **Hocanın vurgusu:** Teoride N değeri 20.000 gibi büyük sayılar alınabilir ama pratikte N ya 1 ya 2'dir; 3 olması bile nadirdir.

Hoca iki sebep verdi:

- Geçmişteki 20.000 karaktere bakan bir model ancak 20.001. karaktere gelince bu geçmişi kullanabilir. İlk 20.000 karakter ASCII ile kodlanmak zorunda kalır; o zaman sıkıştırma olmaz.
- Kombinasyon sayısı çok büyür. ASCII 7 bitliktir ve 128 farklı sembol yazılabilir. Order 2 alınırsa 128^2 farklı ikili, order 3 alınırsa $128^3 = 2.097.152$ (iki milyon küsur) farklı üçlü çıkar. Bunların olasılıkları bir yerde tutulmalı ve bunlara hızlı erişilmelidir; farklı veri yapıları gerekir.

Hoca üçlülere örnek olarak bir metindeki ardışık karakter üçlülerini sıraladı (her üçlünün ve ondan sonra gelen karakterin olasılığının çıkarılması gerektiğini gösterdi).

> **Ders sonrası not:** Hoca order 2 için "1684 tane farklı ikili" dedi; $128^2 = 16.384$'tür (python ile doğrulandı).

Hoca, gerçek uygulamalarda order 1 ya da 2 seçildiğini, ama örneklerinde alfabe 3-4 karakterden oluştuğu için order'ı artıracağını söyledi.

16.2 Context tablosunun kurulması

Örnekte order 4 alınmıştır: bir sembole, geçmişindeki 4 karaktere bakılarak olasılık atanır. Input `xyzzxyxyzzx`'tir (ekrandan, 02:30:00). Hoca tablonun nasıl çıkarıldığını sembol sembol gösterdi:

- İlk x geldi: geçmişi yok. Order 0'a x yazılır, sayısı 1.
- y geldi: order 0'da y 1. y'nin bir karakterlik geçmişi var, x; order 1'de x→y 1.
- z geldi: order 0'da z 1; order 1'de y→z 1; order 2'de xy→z 1.
- İkinci z geldi: order 0'da z 2; order 1'de z→z 1; order 2'de yz→z 1; order 3'te xyz→z 1.
- İkinci x geldi: order 0'da x 2; order 1'de z→x 1; order 2'de zz→x 1; order 3'te yzz→x 1; order 4'te xyzz→x 1. Order 5 yoktur.
- Böyle devam edilir. Beşinci sembolden itibaren order 4'e kadar her order güncellenir.

Sonuç tablosu (a):

```
Order 4      Order 3     Order 2     Order 1    Order 0
xyzz->x 2    xyz->z 2    xy->z 2     x->y 3     x 4
yzzx->y 1    yzz->x 2      ->x 1     y->z 2     y 3
```

```
zzxy->x 1    zzx->y 1    yz->z 2       ->x 1    z 4
zxyx->y 1    zxy->x 1    zz->x 2     z->z 2
xyxy->z 1    xyx->y 1    zx->y 1       ->x 2
yxyz->z 1    yxy->z 1    yx->y 1
```

Tablodaki "→x 1" gibi satırlar, bir üstteki context'in devamıdır (örneğin order 2'de xy→x 1).

Bir z daha gelince (input xyzzxyxyzzxz) tablo (b) olur: order 0'da z 5'e çıkar; z'den önce x hiç gelmediği için order 1'e x→z 1, order 2'ye zx→z 1, order 3'e zzx→z 1, order 4'e yzzx→z 1 yeni satır olarak eklenir. Eksik olanlar tabloya bu şekilde eklenerek ilerlenir (iki tablo da python ile doğrulandı).

16.3 Olasılık hesabı ve order değiştirme

Tablo çıktıktan sonra iki soru doğar: Böyle bir dörtlü karakterin olasılığı nasıl hesaplanacak ve hangi order'daki olasılığın kullanıldığı nereden bilinecek? Algoritmaya bunları bulabilecek ekler yapılmalıdır.

PPM'in çalışma prensibi şudur: Bazen gelen context'in olasılığı sıfırdır, yani daha önce hiç gelmemiştir. Örneğin elde beş tane z olsun ve son z order 4 modeline göre okunacak olsun. Önce en büyük order'a bakılır, çünkü ne kadar uzun bir eşleşme bulunursa o kadar iyidir:

- Order 4: z ve önünde 4 tane z var mı? Yok. Order 3'e "switch" edilir.
- Order 3: z ve önünde 3 tane z? Yok. Order 2'ye geçilir.
- Order 2: z ve önünde 2 tane z? Yok. Order 1'e geçilir.
- Order 1: z önünde z? Evet var. Bu order'daki olasılık değeri kodlanmak üzere gönderilir.

Order 1'de de bulunmazsa order 0'a bakılır. Orada da sembol yoksa yeni bir sembol demektir; tabloya eklenir ve ona bazı değerler verilir.

Hoca bundan sonra anlatılacakları şöyle sıraladı: switch'leri gösteren eklerin (escape), olasılıkların nasıl oluştuğunun, yeni bir sembol gelirse nasıl hazırlanacağının, kodlamanın, veri yapısında nasıl saklandığının ve diğer versiyonlardaki farkların gösterilmesi. Bunların sonraki derste anlatılacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** PPM'de bir order'dan bir alt order'a geçildiğini kod çözücüye bildirmek için özel bir "escape" sembolü kodlanır. Escape'e ve sembollere atanan olasılıklar aritmetik kodlamayla kodlanır; PPM'in A, B, C gibi versiyonları büyük olasılıkla escape olasılığını farklı biçimlerde hesaplar.

Örnek Sorular

Soru 1 (ekrandan, 00:17:20): 10 inç yüksekliğindeki bir sayfa sıkıştırma yapılmadan standart, fine ve very fine modlarda gönderilirse gönderim süresi ne olur?

Çözüm (derste verildi): $10'' = 254$ mm. Scanline sayıları $254 \times 3{,}85 \approx 978$, $254 \times 7{,}7 \approx 1956$, $254 \times 15{,}4 \approx 3912$. Satır başına 1664 pels ile pels/sayfa yaklaşık 1,627M, 3,255M, 6,510M; süreler 170 sn (2,82 dk), 339 sn (5,65 dk), 678 sn (11,3 dk).

Soru 2 (ekrandan, 00:26:20): Modified Huffman ile 12 beyaz, 76 beyaz, 140 beyaz, 64 siyah ve 2561 siyah pel'lik run'lar nasıl kodlanır?

Çözüm (derste verildi): 12 beyaz `001000`; 76 = 64 + 12 → `11011|001000`; 140 = 128 + 12 → `10010|001000`; 64 siyah = 64 + 0 → `0000001111|0000110111`; 2561 = 2560 + 1 → `000000011111|010`.

Soru 3 (ekrandan, 00:28:30): 3 siyah, 2 beyaz, 2 siyah ve 7 beyaz pel'den oluşan 14 pels'lik bir satır Grup 3 ile nasıl kodlanır?

Çözüm (derste verildi): Satır başına bir beyaz pel eklenir: `1w 3b 2w 2b 7w EOL` → `000111 | 10 | 0111 | 11 | 1111 | EOL`.

Soru 4 (ekrandan, 01:30:00): $a_1 \to [0;0{,}4)$, $a_2 \to [0{,}4;0{,}9)$, $a_3 \to [0{,}9;1)$ iken `a2a2a2a3` aritmetik kodlamayla kodlanırsa sonuç aralığı nedir?

Çözüm (derste verildi): $[0{,}4;0{,}9) \to [0{,}6;0{,}85) \to [0{,}7;0{,}825) \to [0{,}8125;0{,}825)$. Kod `8125`.

Soru 5 (ekrandan, 01:43:30): SWISS MISS tablosu verilmişken $0{,}71753375$ kodu nasıl çözülür?

Çözüm (derste verildi): Her adımda aralık bulunur, sembol yazılır, (kod − low)/range ile yeni kod hesaplanır: S (0,4350675), W (0,350675), I (0,753375), S (0,50675), S (0,0135), boşluk (0,135), M (0,35), I (0,75), S (0,5), S (0). Sonuç `SWISS MISS`.

Soru 6 (ekrandan, 01:50:00): Verilen tabloyla `a2a2a1a3a3` kodlanıp çözüldüğünde neden yalnız `a2a2a1` elde edilir ve sorun nasıl giderilir?

Çözüm (derste verildi): a_3'ün aralığının alt sınırı 0'dır. a_1 çözüldükten sonra değer 0 olur ve kod çözücü durur; sondaki a_3'ler kaybolur. Çözüm: olasılığı çok küçük bir EOF sembolü eklemek (son sembol EOF olur) ya da input'un boyutunu önce output'a yazmak.

Soru 7 (derste sözlü, 02:36:10): `xyzzxyxyzzx` input'u için order 4 PPM context tablosunu oluşturun. (Hoca bunu öğrencilerin bir sonraki dersten önce kendilerinin yapmasını istedi.)

> **Çözüm (derste verilmedi):** Python ile her konumda önceki k karakter context olarak sayıldı ($k = 0, \dots, 4$). Sonuç 16.2'deki tablo (a) ile aynıdır: order 0'da x 4, y 3, z 4; order 1'de x→y 3, y→z 2, y→x 1, z→z 2, z→x 2; order 2'de xy→z 2, xy→x 1, yz→z 2, zz→x 2, zx→y 1, yx→y 1; order 3'te xyz→z 2, yzz→x 2, zzx→y 1, zxy→x 1, xyx→y 1, yxy→z 1; order 4'te xyzz→x 2, yzzx→y 1, zzxy→x 1, zxyx→y 1, xyxy→z 1, yxyz→z 1.

Dersin Toparlaması

Dersin ilk bölümünde faks sıkıştırma ele alındı. ITU-T'nin T4 (Grup 3) ve T6 (Grup 4) standartları, dokümanı scanline'lar halinde tarayıp pels'lere çeviren faks makinelerinde sıkıştırmasız gönderim süresini dakikalardan saniyelere indirir. Grup 3'ün tek boyutlu kodlaması Modified Huffman'dır: siyah ve beyaz run'lar, önceden hazırlanmış Termination (0-63) ve Make-up (64'ün katları) tablolarıyla kodlanır. Her satır EOL ile biter ve başına bir beyaz pel eklenir. İki boyutlu Modified READ kodlamasında satır, üstündeki referans satırına göre Pass, Vertical ve Horizontal modlarıyla kodlanır. Daha iyi sıkıştırır ama hataya açıktır; bu yüzden Grup 3'te belli aralıklarla 1D satıra dönülür.

İkinci bölümde aritmetik kodlama anlatıldı: Bütün input'a $[0, 1)$ aralığında tek bir sayı atanır, her sembol aralığı olasılığıyla orantılı olarak daraltır, kod çözme ise aralığı bulma ve (kod − low)/range işlemiyle yapılır. Alt sınırı 0 olan sembolün sonda kalması sorununa karşı EOF sembolü kullanılır. Pratikte tamsayılı, ötelemeli bir sürüm kullanılır. Son bölümde PPM'e giriş yapıldı: order-N Markov modeline dayanan bu adaptif içerik tabanlı yöntemde context tablosu kuruldu ve büyük order'dan küçüğe geçiş fikri tanıtıldı. Hoca escape kodlarını, olasılık hesabını ve PPM'in versiyonlarını sonraki derste anlatacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.